

Группа	Z3342	К работе допущен	_____
Студент	Погорелко В. А.	Работа выполнена	_____
Преподаватель	Маслов И. Д.	Отчёт принят	_____

Рабочий протокол и отчёт по лабораторной работе №5

[C++ & UNIX]: C++ OOP / PARALLEL

1. Цель работы

Познакомить студента с принципами объектно-ориентированного программирования на примере создания сложной синтаксической структуры. Придумать синтаксис своего персонального мини-языка параллельного программирования, а также реализовать его разбор и вычисление.

2. Задачи

1. (C++ PARALLEL LANG) Создать параллельный язык программирования Требуется создать язык программирования, в котором будет доступна установка следующих команд:

- (a) Установка счетного цикла
- (b) Вывод в консоль
- (c) Вывод в файл в режиме добавления
- (d) Арифметические операции +, -, *, /

Счетный цикл должен поддерживать дальнейшую установку всех остальных поддерживаемых команд. Для реализации задачи использовать технологию объектно-ориентированного программирования в части реализации поддерживаемых команд языка.

В программе должны быть отражены следующие шаги:

- (a) Текстовый ввод команд. Каждая новая строка – это новый набор команд.
 - (b) Ожидание команды на окончание ввода
 - (c) Параллельное исполнение введенных строк (наборов команд). Наборы команд должны исполняться параллельно. В консоли фиксировать время запуска / завершения каждого потока. При выводе информации о времени указывать принадлежность потока к строке (набору команд)
2. (LOG) Результат всех вышеперечисленных шагов сохранить в репозиторий (+ отчет по данной ЛР в папку doc) Фиксацию ревизий производить строго через ветку dev. С помощью скриптов накатить ревизии на stg и на prd. Скрипты разместить в корне репозитория. Также создать скрипты по возврату к виду текущей ревизии (даже если в папке имеются несохраненные изменения + новые файлы).

3. Решение

1. (C++ PARALLEL LANG) Создать параллельный язык программирования

В папке lab_05/src создаем файл myLang.cpp:

myLang.cpp Подключение библиотек и классы консольных команд

```
#include <iostream> \\ basic console
#include <mutex> \\ for sync threads
#include <vector> \\ for vectors of objects
#include <string>
#include <sstream> \\ split strings to threads
#include <fstream> \\ work with files
#include <thread> \\ for creating and control threads
#include <chrono> \\ time measure
#include <memory> \\ work with memory blocks
#include <iomanip> \\ form of time logs

std::mutex global_mutex;

class Command {
public:
    virtual ~Command() = default;
    virtual void execute(int set_id) = 0;
};

class PrintCommand : public Command {
    std::string message;
public:
    PrintCommand(std::string msg) : message(std::move(msg)) {}
    void execute(int set_id) override {
        std::lock_guard<std::mutex> lock(global_mutex);
        std::cout << "[ Набор _ бобра _ № " << \
            \ set_id << " ] [ КОНСОЛЬ ]: " << message << std::endl;
    }
};
```

Класс для создания файла

```
class FileAppendCommand : public Command {
    std::string filename;
    std::string data;
public:
    FileAppendCommand(std::string fname, std::string d) :
        filename(std::move(fname)), data(std::move(d)) {}
    void execute(int set_id) override {
        std::lock_guard<std::mutex> lock(global_mutex);
        std::ofstream file(filename, std::ios::app);
        if (file.is_open()) {
            file << data << "\n";
            std::cout << "[ Набор _ бобра _ № " << set_id << "][
                ФАЙЛ _ БОБРШТЕЙНА ]: Дозаписано _ в " <<
                filename << " " << std::endl;
        } else {
            std::cerr << "[ Набор _ бобра _ № " << set_id << "][
                БОБРОВАЯ _ ОШИБКА ]: Не _ удалось _ открыть _
                файл " << filename << std::endl;
        }
    }
};
```

```

class MathCommand : public Command {
    double operand1;
    double operand2;
    char operation;
public:
    MathCommand(double op1, char op, double op2) : operand1(op1),
        operand2(op2), operation(op) {}
    void execute(int set_id) override {
        double result = 0;
        bool valid = true;
        switch (operation) {
            case '+': result = operand1 + operand2; break;
            case '-': result = operand1 - operand2; break;
            case '*': result = operand1 * operand2; break;
            case '/':
                if (operand2 != 0) result = operand1 / operand2;
                else { valid = false; }
                break;
            default: valid = false;
        }

        std::lock_guard<std::mutex> lock(global_mutex);
        if (valid) {
            std::cout << "[ Набор _ бобра _ № " << set_id << " ] [
                БОБРИФМЕТИКА ]: "
                << operand1 << " " << operation << " " <<
                operand2 << " = " << result << std::endl
                ;
        } else {
            std::cout << "[ Набор _ бобра _ № " << set_id <<
                " ] [ БОБРИФМЕТИКА ]: Ошибка _ бобриногo _ вычисления
                (" << operand1 << operation << operand2 << ") " <<
                std::endl;
        }
    }
};

```

Класс для цикла

```
class LoopCommand : public Command {
    int iterations;
    std::vector<std::shared_ptr<Command>> inner_commands;
public:
    LoopCommand(int count) : iterations(count) {}
    void addCommand(std::shared_ptr<Command> cmd) {
        inner_commands.push_back(cmd);
    }

    void execute(int set_id) override {
        for (int i = 0; i < iterations; ++i) {
            {
                std::lock_guard<std::mutex> lock(global_mutex);
                std::cout << "[ Набор _ бобра _ № " << set_id <<
                    "[ БОБРОЦИКЛ ]: Бобритерация " << i + 1 << "
                    из " << iterations << std::endl;
            }
            for (auto& cmd : inner_commands) {
                cmd->execute(set_id);
            }
        }
    }
};
```

```

void log_time(const std::string& status , int set_id) {

auto now = std::chrono::system_clock::now();
    auto time_t_now = std::chrono::system_clock::to_time_t(now);
    auto ms = std::chrono::duration_cast<std::chrono::
        milliseconds>(now.time_since_epoch()) % 1000;
    std::tm tm_buf;
#ifdef _MSC_VER
    localtime_s(&tm_buf, &time_t_now);
#else
    localtime_r(&time_t_now, &tm_buf);
#endif

    std::lock_guard<std::mutex> lock(global_mutex);
    std::cout << ">>> [" << std::put_time(&tm_buf, "%H:%M:%S") <<
        "." << std::setfill('0') << std::setw(3) << ms.count()
        << "]" Порок _ набора _ бобра _ №" << set_id << " "
        << status << std::endl;
}

void run_set(int set_id , std::vector<std::shared_ptr<Command>>
    commands) {
    log_time(" ЗАПУЩЕН ", set_id);
    for (auto& cmd : commands) {
        cmd->execute(set_id);
    }
    log_time(" ЗАВЕРШЕН ", set_id);
}

```

Создание указателя с подсчетом ссылок + запуск конкретных команд из нужного класса

```
std::shared_ptr<Command> parse_single_command(std::stringstream&
ss) {
    std::string cmd_type;
    if (!(ss >> cmd_type)) return nullptr;

    if (cmd_type == "print_to_screen") {
        std::string msg;
        ss >> msg;
        return std::make_shared<PrintCommand>(msg);
    }
    else if (cmd_type == "file_append") {
        std::string fname, data;
        ss >> fname >> data;
        return std::make_shared<FileAppendCommand>(fname, data);
    }
    else if (cmd_type == "calc") {
        double op1, op2;
        char sign;
        ss >> op1 >> sign >> op2;
        return std::make_shared<MathCommand>(op1, sign, op2);
    }
    return nullptr;
}
```

```

std::vector<std::shared_ptr<Command>> parse_line(const std::
string& line) {
    std::vector<std::shared_ptr<Command>> commands;
    std::stringstream ss(line); std::string token;
    while (ss >> token) {
        if (token == "print_to_screen") {
            std::string msg; ss >> msg;
            commands.push_back(std::make_shared<PrintCommand>(msg
                ));
        }
        else if (token == "file_append") {
            std::string fname, data; ss >> fname >> data;
            commands.push_back(std::make_shared<FileAppendCommand>(
                fname, data));
        }
        else if (token == "calc") {
            double op1, op2; char sign;
            ss >> op1 >> sign >> op2;
            commands.push_back(std::make_shared<MathCommand>(op1 ,
                sign , op2));
        }
        else if (token == "loop") {
            int count; ss >> count;
            auto loop = std::make_shared<LoopCommand>(count);
            std::string bracket;
            if (ss >> bracket && bracket == "[") {
                std::string inner_token;
                std::string sub_cmd_line = "";
                while (ss >> inner_token && inner_token != "]" ) {
                    if (inner_token == ";") {
                        std::stringstream sub_ss(sub_cmd_line);
                        auto sub_cmd = parse_single_command(
                            sub_ss);
                        if (sub_cmd) loop->addCommand(sub_cmd);
                        sub_cmd_line = "";
                    } else {
                        sub_cmd_line += inner_token + " ";
                    }
                }
                if (!sub_cmd_line.empty()) {
                    std::stringstream sub_ss(sub_cmd_line);
                    auto sub_cmd = parse_single_command(sub_ss);
                    if (sub_cmd) loop->addCommand(sub_cmd);
                }
            }
            commands.push_back(loop);
        }
    }
    return commands;
}

```



```
int main() {
    std::setlocale(LC_ALL, "Russian");
    std::vector<std::vector<std::shared_ptr<Command>>>
        program_storage;
    std::string line;

    "INITIAL TEXT"

    int line_counter = 1;
    while (true) {
        std::cout << "Набор" " " << line_counter << " > ";
        std::getline(std::cin, line);
        if (line.empty()) {
            break;
        }
        program_storage.push_back(parse_line(line));
        line_counter++;
    }

    std::cout << "START" << std::endl;

    std::vector<std::thread> workers;
    for (size_t i = 0; i < program_storage.size(); ++i) {
        workers.emplace_back(run_set, static_cast<int>(i + 1),
            program_storage[i]);
    }

    for (auto& thread : workers) {
        if (thread.joinable()) {
            thread.join();
        }
    }

    std::cout << "END" << std::endl;
    std::cout << " Все _ бобровые _ потоки _ завершили _ работу . _
        Программа _ окончена, _ плотина _ построена. " << std::endl;
    return 0;
}
```

INITIAL TEXT

```

std::cout << " ИНТЕРПРЕТАТОР _ ПАРАЛЛЕЛЬНОГО _ ЯП _
BOBER " << std::endl;
std::cout << " Правила _ ввода _ команд _ в( _ одну _ строку _
через _ пробел):\n"
<< " - print_to_screen [word]\n"
<< " - file_append [file.txt] [word]\n"
<< " - calc [num] [operation +, -, *, /] [num]\n"
<< " - loop [iters] [ command_1 ; command_2 ; ] (
need spaces between '[' , ']' and ';' )\n"
<< " Каждая _ строка—новый _ независимый _ набор.\n
"
<< " Для _ завершения _ ввода _ нажмите _ ENTER на _
пустой _ строке.\n" << std::endl;

```